

Data Structures Notes: Linked Lists

1. Introduction to Linked Lists

A **linked list** is a dynamic linear data structure in which elements are stored in separate memory locations and connected using **pointers**.

Unlike an array, linked-list elements do **not need to occupy contiguous memory locations**.

Each element is called a **node**.

A basic singly linked-list node contains:

1. **Data** – stores the actual value.
2. **Link/Next** – stores the address of the next node.

Structure of a node

```
+-----+-----+
|  DATA  |  NEXT  |
+-----+-----+
```

For example:

```
HEAD
|
v
+---+---+      +---+---+      +---+---+
| 10 | *----->| 20 | *----->| 30 | NULL |
+---+---+      +---+---+      +---+---+
```

Here:

- **HEAD** points to the first node.
- Each **NEXT** contains the address of the next node.
- The last node contains **NULL**.
- **NULL** indicates the end of the list.

2. Singly Linked List

A **singly linked list** is a linked list in which each node contains a pointer to **only the next node**.

General representation

```
HEAD → NODE1 → NODE2 → NODE3 → ... → NODEn → NULL
```

For example:

```
HEAD
↓
[10 | 2000] → [20 | 3500] → [30 | 4500] → [40 | NULL]
```

The numbers such as 2000, 3500, and 4500 represent memory addresses.

3. Representation in Memory

In an array, elements are normally stored in consecutive memory locations:

Address	Data
1000	10
1004	20
1008	30
1012	40

In a linked list, nodes can be located anywhere in memory:

Address 1000	Address 2500
+-----+-----+	+-----+-----+
10 2500 ---->	20 7000
+-----+-----+	+-----+-----+

Address 7000
+-----+-----+
30 NULL
+-----+-----+

Therefore:

Linked-list nodes need not be stored in contiguous memory.

The `NEXT` field maintains the logical order.

Advantages of memory representation

- Dynamic memory allocation
- Efficient insertion and deletion
- No need to know the size in advance

- Memory is allocated as required

Disadvantages

- Extra memory is required for pointers.
 - Direct/random access is not possible.
 - Searching requires sequential traversal.
 - Pointer manipulation makes implementation more complex.
-

4. Declaration of a Singly Linked List Node in C

C

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

A pointer to the first node can be declared as:

C

```
struct Node *head = NULL;
```

Initially:

HEAD → NULL

5. Creating a New Node

Dynamic memory allocation is generally used.

C

```
struct Node *newNode;  
  
newNode = (struct Node *)malloc(sizeof(struct Node));  
  
newNode->data = 10;  
newNode->next = NULL;
```

The `malloc()` function allocates memory dynamically.

Header file:

C

```
#include <stdlib.h>
```

6. Basic Operations on Singly Linked List

The important operations are:

1. Traversing
2. Searching
3. Insertion
4. Deletion

7. Traversing a Linked List

Meaning

Traversal means visiting every node of the linked list exactly once.

Suppose:

HEAD

↓

10 → 20 → 30 → 40 → NULL

Traversal produces:

10 20 30 40

Algorithm: Traversing

Algorithm TRAVERSE(HEAD)

1. Start.
2. Set `PTR = HEAD`.
3. Repeat while `PTR != NULL` :
 - Display `PTR->DATA`.
 - Set `PTR = PTR->NEXT`.
4. Stop.

C function

C

```
void traverse(struct Node *head)
{
    struct Node *ptr = head;

    while (ptr != NULL)
    {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
}
```

Complexity

If there are n nodes:

Time Complexity = $O(n)$

Space Complexity = $O(1)$

8. Searching in a Linked List

Meaning

Searching means finding whether a particular value exists in the linked list.

Example:

10 → 20 → 30 → 40 → NULL

Search for 30 .

Comparison:

30 == 10 → No
30 == 20 → No
30 == 30 → Yes

Therefore, 30 is found.

Algorithm: Searching

Algorithm SEARCH(HEAD, ITEM)

1. Start.
2. Set `PTR = HEAD`.
3. Repeat while `PTR != NULL`:
 - If `PTR->DATA == ITEM`, return **FOUND**.
 - Otherwise set `PTR = PTR->NEXT`.
4. If `PTR == NULL`, return **NOT FOUND**.
5. Stop.

C function

C

```
int search(struct Node *head, int item)
{
    struct Node *ptr = head;

    while (ptr != NULL)
    {
        if (ptr->data == item)
            return 1;

        ptr = ptr->next;
    }

    return 0;
}
```

Complexity

Case	Complexity
Best case	O(1)
Average case	O(n)
Worst case	O(n)

9. Insertion into a Singly Linked List

Insertion means adding a new node to the linked list.

Common insertion operations are:

1. Insertion at beginning

2. Insertion at end
3. Insertion after a given node

9.1 Insertion at Beginning

Original:

```
HEAD
↓
10 → 20 → 30 → NULL
```

Insert 5 .

New list:

```
HEAD
↓
5 → 10 → 20 → 30 → NULL
```

Steps

1. Create a new node.
2. Store data in the new node.
3. Make the new node point to `HEAD` .
4. Make `HEAD` point to the new node.

Algorithm

Algorithm INSERT_BEGINNING(HEAD, ITEM)

1. Create a new node `NEW` .
2. If memory is unavailable, report overflow.
3. Set `NEW->DATA = ITEM` .
4. Set `NEW->NEXT = HEAD` .
5. Set `HEAD = NEW` .
6. Stop.

C function

C

```
void insertBeginning(struct Node **head, int item)
{
    struct Node *newNode;
```

```

newNode = (struct Node *)malloc(sizeof(struct Node));

newNode->data = item;
newNode->next = *head;

*head = newNode;
}

```

Complexity

Time = $O(1)$

10. Insertion at End

Original:

```

HEAD
↓
10 → 20 → 30 → NULL

```

Insert 40 .

```

HEAD
↓
10 → 20 → 30 → 40 → NULL

```

Steps

1. Create a new node.
2. Set its data.
3. Set its `next = NULL` .
4. Traverse to the last node.
5. Set the last node's `next` to the new node.

Algorithm

Algorithm INSERT_END(HEAD, ITEM)

1. Create `NEW` .
2. Set `NEW->DATA = ITEM` .
3. Set `NEW->NEXT = NULL` .
4. If `HEAD == NULL` , set `HEAD = NEW` and stop.
5. Set `PTR = HEAD` .
6. Repeat while `PTR->NEXT != NULL` :

- `PTR = PTR->NEXT`

7. Set `PTR->NEXT = NEW` .

8. Stop.

C function

C

```
void insertEnd(struct Node **head, int item)
{
    struct Node *newNode;
    struct Node *ptr;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = item;
    newNode->next = NULL;

    if (*head == NULL)
    {
        *head = newNode;
        return;
    }

    ptr = *head;

    while (ptr->next != NULL)
        ptr = ptr->next;

    ptr->next = newNode;
}
```

Complexity

Without a tail pointer:

Time = $O(n)$

With a tail pointer:

Time = $O(1)$

11. Insertion After a Given Node

Suppose:

10 → 20 → 40 → NULL

Insert 30 after 20 .

Result:

10 → 20 → 30 → 40 → NULL

Pointer changes

Before:

20 → 40

After:

20 → 30 → 40

C function

C

```
void insertAfter(struct Node *prev, int item)
{
    struct Node *newNode;

    if (prev == NULL)
        return;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = item;
    newNode->next = prev->next;

    prev->next = newNode;
}
```

Complexity

If the required node is already available:

Time = $O(1)$

If it must first be searched:

Time = $O(n)$

12. Deletion from a Singly Linked List

Deletion means removing a node from the linked list.

Common deletion operations are:

1. Delete from beginning
2. Delete from end
3. Delete a specific node/value

13. Deletion from Beginning

Original:

```
HEAD
↓
10 → 20 → 30 → NULL
```

Delete the first node.

Result:

```
HEAD
↓
20 → 30 → NULL
```

Steps

1. Check whether the list is empty.
2. Store `HEAD` in a temporary pointer.
3. Move `HEAD` to the next node.
4. Free the old first node.

Algorithm

Algorithm DELETE_BEGINNING(HEAD)

1. If `HEAD == NULL`, report **UNDERFLOW**.
2. Set `TEMP = HEAD`.
3. Set `HEAD = HEAD->NEXT`.
4. Free `TEMP`.
5. Stop.

C function

C

```
void deleteBeginning(struct Node **head)
{
    struct Node *temp;

    if (*head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = *head;
    *head = (*head)->next;

    free(temp);
}
```

Complexity

Time = $O(1)$

14. Deletion from End

Original:

10 → 20 → 30 → NULL

Delete 30 .

Result:

10 → 20 → NULL

To delete the last node, we need to reach the **second-last node**.

Algorithm

Algorithm DELETE_END(HEAD)

1. If `HEAD == NULL` , report underflow.
2. If `HEAD->NEXT == NULL` :
 - Free `HEAD` .
 - Set `HEAD = NULL` .
 - Stop.

3. Set `PTR = HEAD` .
4. Repeat while `PTR->NEXT->NEXT != NULL` :
 - `PTR = PTR->NEXT`
5. Free `PTR->NEXT` .
6. Set `PTR->NEXT = NULL` .
7. Stop.

C function

C

```
void deleteEnd(struct Node **head)
{
    struct Node *ptr;
    struct Node *temp;

    if (*head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    if ((*head)->next == NULL)
    {
        free(*head);
        *head = NULL;
        return;
    }

    ptr = *head;

    while (ptr->next->next != NULL)
        ptr = ptr->next;

    temp = ptr->next;
    ptr->next = NULL;

    free(temp);
}
```

Complexity

Time = $O(n)$

15. Deletion of a Specific Node

Suppose:

10 → 20 → 30 → 40 → NULL

Delete 30 .

Result:

10 → 20 → 40 → NULL

The link of 20 must be changed:

20 → 30 → 40

becomes:

20 → 40

Then node 30 is freed.

C function

C

```
void deleteValue(struct Node **head, int item)
{
    struct Node *ptr;
    struct Node *temp;

    if (*head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    if ((*head)->data == item)
    {
        temp = *head;
        *head = (*head)->next;
        free(temp);
        return;
    }

    ptr = *head;

    while (ptr->next != NULL &&
           ptr->next->data != item)
    {
        ptr = ptr->next;
    }
}
```

```

    if (ptr->next == NULL)
    {
        printf("Item not found\n");
        return;
    }

    temp = ptr->next;
    ptr->next = temp->next;

    free(temp);
}

```

Complexity

Time = $O(n)$

16. Linked List Operations: Complexity

Operation	Time Complexity
Traversal	$O(n)$
Search	$O(n)$
Insert at beginning	$O(1)$
Insert at end	$O(n)$
Insert after known node	$O(1)$
Delete beginning	$O(1)$
Delete end	$O(n)$
Delete known node	$O(1)$
Delete searched node	$O(n)$

Important point

Linked lists are particularly useful when **frequent insertion and deletion** are required.

17. Array vs Linked List

Feature	Array	Linked List
Memory	Usually contiguous	Non-contiguous
Size	Usually fixed/static	Dynamic
Random access	$O(1)$	$O(n)$
Insertion at beginning	$O(n)$	$O(1)$
Deletion at beginning	$O(n)$	$O(1)$
Extra pointer memory	No	Yes
Memory utilization	May have unused allocated space	Allocated as needed
Implementation	Simpler	More complex

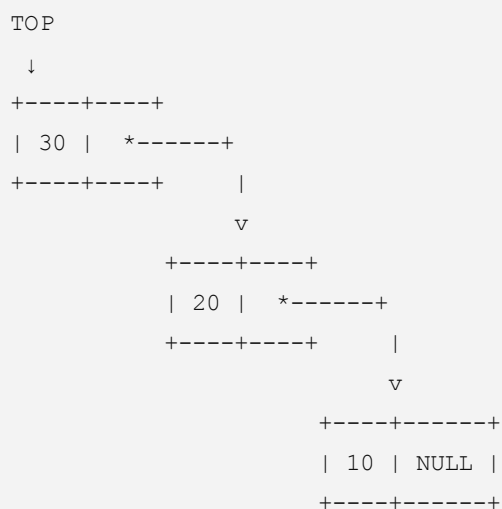
18. Linked Representation of Stack

A **stack** follows:

LIFO — Last In, First Out

A stack can be implemented using a linked list.

The **top** pointer points to the first node.



Here:

TOP → 30 → 20 → 10 → NULL

19. Push Operation

Push means inserting an element into the stack.

For a linked stack, insertion is normally performed at the beginning.

Before:

TOP → 20 → 10 → NULL

Push 30 :

TOP → 30 → 20 → 10 → NULL

Algorithm PUSH(TOP, ITEM)

1. Create a new node.
2. If memory is unavailable, report overflow.
3. Store `ITEM` in the new node.
4. Set `NEW->NEXT = TOP`.
5. Set `TOP = NEW`.
6. Stop.

C function

C

```
void push(struct Node **top, int item)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Stack Overflow\n");
        return;
    }

    newNode->data = item;
    newNode->next = *top;
```

```
*top = newNode;
```

Time = $O(1)$

20. Pop Operation

Pop means removing the top element.

Before:

```
TOP → 30 → 20 → 10 → NULL
```

Pop:

```
TOP → 20 → 10 → NULL
```

Algorithm POP(TOP)

1. If `TOP == NULL`, report **UNDERFLOW**.
2. Set `TEMP = TOP`.
3. Set `TOP = TOP->NEXT`.
4. Free `TEMP`.
5. Stop.

C function

C

```
void pop(struct Node **top)
{
    struct Node *temp;

    if (*top == NULL)
    {
        printf("Stack Underflow\n");
        return;
    }

    temp = *top;
    *top = (*top)->next;

    printf("Deleted: %d\n", temp->data);

    free(temp);
}
```

Time = $O(1)$

21. Linked Representation of Queue

A **queue** follows:

FIFO — First In, First Out

For a linked queue, two pointers are normally maintained:

- **FRONT** — points to the first node.
- **REAR** — points to the last node.



22. Enqueue Operation

Enqueue means inserting an element at the rear.

Before:

FRONT → 10 → 20 → 30 → NULL ← REAR

Enqueue 40 :

FRONT → 10 → 20 → 30 → 40 → NULL
↑
REAR

Algorithm ENQUEUE(FRONT, REAR, ITEM)

1. Create a new node.
2. If memory is unavailable, report overflow.
3. Set **NEW->DATA = ITEM**.
4. Set **NEW->NEXT = NULL**.
5. If **REAR == NULL** :
 - **FRONT = REAR = NEW**
6. Otherwise:

- REAR->NEXT = NEW
- REAR = NEW

7. Stop.

C function

C

```
void enqueue(struct Node **front,
             struct Node **rear,
             int item)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Queue Overflow\n");
        return;
    }

    newNode->data = item;
    newNode->next = NULL;

    if (*rear == NULL)
    {
        *front = *rear = newNode;
    }
    else
    {
        (*rear)->next = newNode;
        *rear = newNode;
    }
}
```

Time = $O(1)$

23. Dequeue Operation

Dequeue means removing an element from the front.

Before:

FRONT → 10 → 20 → 30 → NULL ← REAR

Dequeue:

FRONT → 20 → 30 → NULL ← REAR

Algorithm DEQUEUE(FRONT, REAR)

1. If FRONT == NULL , report UNDERFLOW.
2. Set TEMP = FRONT .
3. Set FRONT = FRONT->NEXT .
4. If FRONT == NULL , set REAR = NULL .
5. Free TEMP .
6. Stop.

C function

C

```
void dequeue(struct Node **front,
             struct Node **rear)
{
    struct Node *temp;

    if (*front == NULL)
    {
        printf("Queue Underflow\n");
        return;
    }

    temp = *front;
    *front = (*front)->next;

    if (*front == NULL)
        *rear = NULL;

    printf("Deleted: %d\n", temp->data);

    free(temp);
}
```

Time = O(1)

24. Stack vs Queue Using Linked List

Feature	Linked Stack	Linked Queue
Principle	LIFO	FIFO

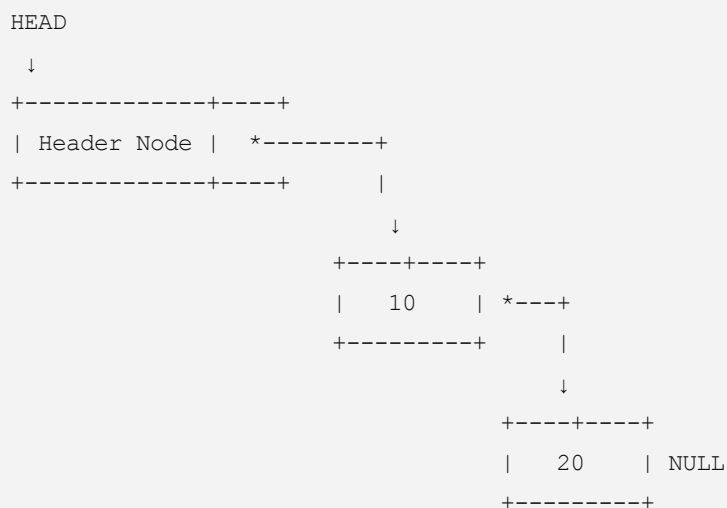
Feature	Linked Stack	Linked Queue
Main pointer(s)	TOP	FRONT, REAR
Insertion	TOP	REAR
Deletion	TOP	FRONT
Insertion complexity	O(1)	O(1)
Deletion complexity	O(1)	O(1)
Underflow	TOP = NULL	FRONT = NULL

25. Header Node

A **header node** is a special node placed at the beginning of a linked list.

It generally does **not represent an ordinary data element**.

Example:



The header node can store:

- Metadata
- Number of nodes
- A pointer to the first actual node
- Other information required by the application

26. Ordinary List vs Header List

Ordinary linked list

```
HEAD → [10] → [20] → [30] → NULL
```

Header linked list

```
HEAD → [HEADER] → [10] → [20] → [30] → NULL
```

The header node is **not normally considered part of the data elements**.

27. Advantages of Header Nodes

Header nodes can simplify linked-list operations.

1. Simplifies insertion

Special cases involving the first data node can sometimes be avoided.

2. Stores metadata

For example:

```
HEADER
├─ count = 3
└─ next → first node
```

3. Simplifies algorithms

Operations can be written more uniformly because a node always exists before the first actual data node.

4. Useful for special linked lists

Header nodes are commonly used in:

- Polynomial representation
 - Sparse matrices
 - Circular linked lists
 - General linked-list implementations
-

28. Important Terms

Term	Meaning
Node	Individual element of a linked list
Data field	Stores the actual value
Link field	Stores address of another node
HEAD	Pointer to first node
NEXT	Pointer to next node
NULL	Indicates no next node
Traversal	Visiting all nodes
Insertion	Adding a node
Deletion	Removing a node
Search	Finding a particular value
TOP	Pointer to top of linked stack
FRONT	Pointer to first queue node
REAR	Pointer to last queue node
Header node	Special node at beginning of a list

29. Important Exam Points

Singly Linked List

A singly linked list is a collection of nodes where each node contains:

DATA + POINTER TO NEXT NODE

The last node points to:

NULL

Traversal


```
HEAD → NODE → NODE → NODE → NULL
                ↑
            stop here
```

Insertion at beginning

```
NEW → OLD HEAD
HEAD = NEW
```

Deletion at beginning

```
TEMP = HEAD
HEAD = HEAD->NEXT
FREE (TEMP)
```

Stack using linked list

```
TOP → NODE → NODE → NODE → NULL
```

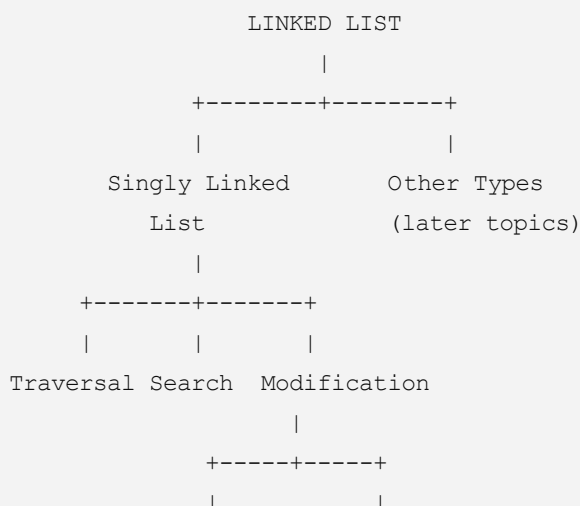
Both **PUSH** and **POP** are **O(1)**.

Queue using linked list

```
FRONT → NODE → NODE → NODE ← REAR
```

Both **ENQUEUE** and **DEQUEUE** are **O(1)** when **FRONT** and **REAR** pointers are maintained.

30. Overall Concept Map



```

                Insertion      Deletion
                  |
          +-----+-----+
          |           |           |
        Beginning   End     After Node

Linked List Applications
    |
  +---+---+
  |       |
Stack    Queue
  |       |
TOP     FRONT/REAR
  |       |
PUSH/POP ENQUEUE/
         DEQUEUE

Special Concept
    |
Header Node

```

Quick revision

```

Singly Linked List
    ↓
Node = DATA + NEXT
    ↓
HEAD points to first node
    ↓
Last node → NULL
    ↓
Traversal/Search → O(n)
    ↓
Insert beginning → O(1)
Delete beginning → O(1)
    ↓
Stack → TOP → PUSH/POP
    ↓
Queue → FRONT/REAR → ENQUEUE/DEQUEUE
    ↓
Header Node → special first node

```

Key idea: A linked list trades the array's fast random access for **dynamic memory allocation and efficient pointer-based insertion/deletion**.